

```
> h.next();
End
Object { value: undefined, done: true }
```

Automatic creation of an iterable object

The code example given below shows the Fibonacci programme implemented using a generator:

```
1. // generator function
2. function* fibonacci() {
3.   let [prev, curr] = [0, 1];
4.   for (;;) {
5.     [prev, curr] = [curr, prev + curr];
6.     yield curr;
7.   }
8. }
9.
10. //controller
11. function controller() {
12.   for (let f of fibonacci()) {
13.     if (f > 1000) return;
14.     console.log(f);
15.   }
16. }
17. controller();
```

Control switches between the generator and the controller. In the above example, you will notice that the generator returns an iterable object and hence can be used in a 'for..of' loop. This is one of the benefits of using a generator. In the previous article on iterators, we learnt that they can be used to create custom iterators, which require careful programming, as they need to maintain the internal state. With generators, the creation of the iterator is implicit and state is maintained.

Passing values to generators and yield

The following example shows the passing of a value to a generator function, just like any other function. It also illustrates the passing of the value from the `next()` method. This allows the passing of values between generator and controlling code.

```
1. function *calc(a) {
2.   var plus1 = yield (a + 1);
3.   var div2 = yield (plus1 / 2);
4.   return (a + plus1 + div2);
5. }
6.
7. var iter = calc(1);
8. console.log(iter.next());
```

```
9. console.log(iter.next(2));
10. console.log(iter.next(3));
11. The output is:
12. { value: 2, done: false }
13. { value: 1, done: false }
14. { value: 6, done: true }
```

Asynchronous programming

In JavaScript, there was only a callback mechanism which allowed asynchronous programming. The generators feature takes asynchronous programming to an entirely new level. In the above example, we have seen the keyword 'yield' followed by an expression. The keyword 'yield' can be followed by async functions such as file fetch, URL fetch or database operations.

Compute and storage

Generators help in processing collections or streaming of data, without the need to collect all data. They allow incremental processing as well as lazy evaluation, unfortunately. Lazy evaluation is related to functional programming and is beyond the scope of this article. Programming using generators is preferred when compute is cheaper than storage.

Support matrix

Support for generators is great in the latest versions of Google Chrome, Microsoft Edge and Node.js. The Firefox browser, too, has generator functionality to the greatest extent.

Language continues to evolve

In the last 10 articles in this series, we learnt about most of the important features of ES6. There are many variations and permutations possible with its features. It is left to programmers to explore and exploit the features to write better and more maintainable code. So, continue to use the features and do contribute to standards bodies to make JavaScript the best programming language. 

References

- [1] Understanding JavaScript generators by James Moore <https://www.youtube.com/watch?v=YrZyAn0yCb8>
- [2] Hanging Up on Callbacks: Generators in ECMAScript 6 <http://es6-gen.herokuapp.com/index.html>
- [3] What's the Big Deal with Generators <http://devsmash.com/blog/whats-the-big-deal-with-generators>

By: Janardan Revuru

The author works as an Engineering Manager at Hewlett Packard Enterprise, Bengaluru. He is a co-organiser of the 'JavaScript meetup-Bangalore' group at [meetup.com](http://www.meetup.com/JavaScript-Meetup-Bangalore/) (<http://www.meetup.com/JavaScript-Meetup-Bangalore/>).